

constexpr if

Changes from previous version

- This paper revises P0128R0.
- Added a `get()` example.
- Replaced the new `constexpr_if` keyword by the sequence of existing keywords `constexpr if`.
- Added proposed wording.

Abstract

I want to bring back parts of static if; namely bring it back in a form where it's

- restricted to block scopes.
- restricted to appear only within templates.
- always going to establish a new scope.
- required that there exists values of the condition so that either condition branch is well-formed.

Why? Because it allows making static decisions without having to resort to multiple overloads. Having a static if allows for simple and local code, without having to know the intricacies of overload resolution, partial ordering and SFINAE.

Introduction

Richard Smith explained the following:

```
The "controversial" parts of N3329 are that:
1) it does not introduce a new scope, and
2) the non-selected branch is completely ignored (the tokens aren't even
required to be parseable)
```

```
This makes it fundamentally incompatible with the template model used by at
least two major implementations.
```

```
If, instead, it introduced a new scope (as proposed in this thread) and we had
a requirement that it is possible to instantiate each arm of the static if
(that is, the same requirement we have for other token sequences in templates),
then I believe the over-my-dead-body objections from implementors would disappear.
```

So, the proposed `constexpr if` statement should have the characteristics Richard outlined.

Motivation

As the first example, I find it unwieldy to do pack unpacking with multiple overloads.

```
template <class T>
void f(T&& t)
{
    /* handle one T */
}

template <class T, class... Rest>
void f(T&& t, Rest&&... r)
{
    f(t);
    /* handle the tail */
    f(r...); // I think I have a bug here if I don't have a zero-param overload
}
```

It would be much simpler to be able to handle the unpacking in one function template, even though we're still writing recursive

code.

```
template <class T, class... Rest>
void f(T&& t, Rest&&... r)
{
    /*
     * handle one T
     */
    constexpr if (sizeof...(r)) {
        /*
         * handle the tail
         */
        f(r...); // I don't need a zero-param overload to do this
    }
}
```

Mutually exclusive constraints would also be arguably easier to grok. Instead of

```
template <class T, class... Args>
enable_if_t<is_constructible_v<T, Args...>, unique_ptr<T>>
make_unique(Args&&... args)
{
    return unique_ptr<T>(new T(forward<Args>(args)...));
}
```

```
template <class T, class... Args>
enable_if_t<!is_constructible_v<T, Args...>, unique_ptr<T>>
make_unique(Args&&... args)
{
    return unique_ptr<T>(new T{forward<Args>(args)...});
}
```

we could write

```
template <class T, class... Args>
unique_ptr<T>
make_unique(Args&&... args)
{
    constexpr if (is_constructible_v<T, Args...>) {
        return unique_ptr<T>(new T(forward<Args>(args)...));
    } constexpr_else {
        return unique_ptr<T>(new T{forward<Args>(args)...});
    }
}
```

Even if the `enable_if`s above are turned into constraints, I daresay the single-function solution is much simpler. A "damn sight nicer", if you ask me.

I expect there are many more good uses for such a facility than I can imagine. I have heard users hinting at wanting to write a function template that can take both signed and unsigned integral types, and write different code for the signed and unsigned cases, without having to worry about either branch emitting diagnostics even if never being taken - and those users do not think they want to write multiple overloads for integral types, since getting something like that right may end up being a heroic endeavor...

Can't we do this with existing language features?

John Spicer suggested in [c++std-ext-17099](#) that polymorphic lambdas combined with a decision-making template would provide an adequate facility without a need to add new language features. The invocation of that decision-making template looks roughly like this:

```
template <int arg, typename ... Args> int do_something(Args... args) {
    return static_if<sizeof...(args)>::get(
        [](auto x, auto y) { return x+y; },
        [](auto x) { return *x; })(args...);
}
```

Now, in comparison, with the proposed language facility, we do

```
template <int arg, typename ... Args> int do_something(Args... args) {
    constexpr if (sizeof...(args)) {
        return (args + ...);
    } constexpr_else {
        return *args...;
    }
}
```

Yes, I'm cheating - we nowadays have fold expressions. :) Without them, the equivalent code without would probably be written with a lambda, or by using a temporary tuple or array. I must point out some things here:

- I can return, break, continue and goto from within a constexpr if block. I can not do that from within the lambda.
- While I am a big proponent of using lambdas to create new control facilities, I find the constexpr if solution infinitely more readable.

Richard Smith explained the following:

```
Right, when a function template is instantiated, all of the
declarations/statements/expressions within it are instantiated,
and that includes pieces inside local classes, generic lambdas, and so on.
```

```
This instantiation of generic lambda bodies is in fact necessary for
our language semantics -- computing the captures of a generic lambda
within a function template specialization relies on us having already
instantiated the complete closure type and its call operator template
to the point that we know where the odr-uses are within the non-dependent
full-expressions within the body.
```

In contrast, the intent of constexpr if is that the branch not taken is not instantiated.

Yes, but don't Concepts provide a superior alternative?

Yes, for expressing the constraints of a function template, they do. No, for simplicity and locality of code, they don't. It's certainly easier to write (mutually exclusive and other) constraints with concepts, since it's possible to overload on concepts. The lack of locality remains, and the need to understand overload resolution, partial ordering and SFINAE remains. I posit that there are many simple cases where all that is still overly complex when a simple block-scope static condition would do much better. Chances are, of course, that combining Concepts with a constexpr if can lead to expressive designs that are far superior to what either of these facilities can provide in isolation.

Example for tying into a potential destructuring facility

Somewhat hypothetically, we might one day get a facility that allows destructuring compound objects into individual objects. There's some chance that something like that would be implemented in terms of a get<> function or function template. With constexpr if, I believe I can write the following:

```
class X
{
    int x;
    string y;
    float z;
public:
    int get_x() {return x;}
    string get_y() {return y;}
    float get_z() {return z;}
};

template <size_t I>
auto get(X& x)
{
    constexpr if (I == 0)
        return x.get_x();
    constexpr if (I == 1)
        return x.get_y();
    constexpr if (I == 2)
        return x.get_z();
    // static_assert(I <= 2); // not needed in this particular case
}
```

Incidentally, this code relies on the fact that `void` is NOT a regular type; it is assumed that the destructuring facility that calls it will initialize variables of the return type, or of references to the return type. Therefore the calling code will automatically be ill-formed if the function above is called with `I` larger than two, since the return type is `void`, and variables of type `void` or reference to `void` are forbidden.

Now, let's write the `get` part in C++14 code:

```
template <size_t I>
auto get(X& x);

template <>
auto get<0>(X& x)
{
    return x.get_x();
}

template <>
auto get<1>(X& x)
{
    return x.get_y();
}

template <>
auto get<2>(X& x)
{
    return x.get_z();
}
```

You might say "oh, we need to deal with rvalues, too". I can certainly easily write the one overload for the function using a `constexpr` if. And sure, I can write the three additional overloads for the rvalue case in C++14. Which one is more palatable is left as an exercise for the reader, but to me, the decision which technique to use would be somewhat obvious. I can also write a template that handles both lvalues and rvalues, or three template overloads for the C++14 case.

Implementability

Faisal Vali has implemented a prototype for clang, [here](#).

Teachability

Adding this facility will increase the overall complexity of the language, and since it's not identical or even very similar to the static if in D, it's not trivial to teach. I do have high hopes that it would be much simpler to teach for simple cases than using multiple overloads or using a metaprogramming facility with lambdas would be.

Wording

In [stmt.select], edit the grammar for *selection-statement* as follows:

```
selection-statement:
    if ( condition ) statement
    if ( condition ) statement else statement
    constexpr if ( condition ) statement
    constexpr if ( condition ) statement constexpr else statement
    switch ( condition ) statement
```

Insert a new subclause after [stmt.if]:

6.4.2 The `constexpr if` statement

A `constexpr if` statement shall appear only in the *function-body* of a function template or in the *compound-statement* of a generic lambda. The *condition* shall be a contextually converted constant expression of type `bool`. During an instantiation of the enclosing function template or generic lambda, if the converted condition is `true` and the statement includes a `constexpr else` substatement, that substatement is not instantiated. Conversely, if the condition is `false`, the first substatement is not instantiated. A label (`stmt.label`) declared in a substatement that is not instantiated shall not be referred to by a statement (`stmt.goto` , `stmt.switch`) that is instantiated.

[Example:

```
template<typename T, typename ... Rest > void g(T&& p, Rest&& ...rs) {  
    // ... handle p  
    constexpr if (sizeof...(rs) > 0) {  
        f(rs...); // Never instantiated with an empty argument list.  
    }  
}
```

— end example]

In all other respects, a constexpr if statement is equivalent to the ordinary if statement obtained by dropping the leading constexpr keyword.

Edit [temp.inst]/11 as follows:

An implementation shall not implicitly instantiate a function template, a variable template, a member template, a non-virtual member function, a member class, ~~or~~ a static data member of a class template, or a substatement of a constexpr if statement that does not require instantiation.

Edit [temp.decls]/2 as follows:

For purposes of name lookup and instantiation, default arguments and exception-specifications of function templates and default arguments and exception-specifications of member functions of class templates are considered definitions; each default argument or exception-specification is a separate definition which is unrelated to the function template definition or to any other default arguments or exception-specifications. For the purpose of instantiation, the substatements of a constexpr if statement are considered definitions; each such statement is a separate definition which is unrelated to the surrounding function definition.